A Database Management System Project Report on

BUS RESERVATION SYSTEM

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By:

Avinash MK - 2501010040
Shyam Nath Patro - 2501010130
Arijit Deb - 2501010037
Shaswat Dwivedi - 2501010090
Priyanshu Singh - 2501010017

Under the Guidance of:

Rishav Upadhyay

[PW Institue of Innovation , Medhavi Skills University]

[School of Technology]

Semester: 2nd

Academic Year: 2026

CERTIFICATE

This is to certify that the Database Management System project report entitled **"Bus Reservation System"** is a bonafide record of the work carried out by **the Students** under my supervision and guidance, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering from **PW Institute of Innovation** and **Medhavi Skills University**

The work embodied in this report is original and has not been submitted to any other University or Institution for the award of any degree or diploma.

_____

Rishav Upadhyay

ABSTRACT

The Bus Reservation System is a comprehensive, database-driven web application meticulously designed to automate and streamline the management and ticketing operations of a bus transport network. As the transportation industry experiences rapid growth, the reliance on manual ledgers and fragmented records has proven inefficient, error-prone, and difficult to scale. This project addresses these challenges by providing a robust digital infrastructure.

The system is built upon a solid backend architecture utilizing a normalized MySQL relational database to guarantee data integrity, eliminate redundancy, and efficiently manage complex interactions between core entities: Passengers, Buses, Conductors, and Tickets. The backend server is engineered using Node.js and the native HTTP module, exposing a RESTful API to handle database transactions asynchronously.

The frontend consists of a responsive, modern web interface built with HTML, CSS, and vanilla JavaScript. It empowers administrators to perform sophisticated CRUD (Create, Read, Update, Delete) operations effortlessly. Features include a dynamic dashboard summarizing real-time metrics, comprehensive fleet and crew management modules, ticket generation, and advanced reporting capabilities powered by complex SQL Joins and Views.

By enforcing strict referential integrity and leveraging parameterized queries, the system ensures both operational reliability and protection against standard vulnerabilities. Ultimately, the Bus Reservation System provides a scalable, user-friendly, and highly consistent platform that significantly enhances administrative convenience and operational logistics.

TABLE OF CONTENTS

1. INTRODUCTION

The advent of digital technologies has revolutionized various industrial sectors, and the transportation industry is no exception. With globalization and the increasing mobility of the workforce and general population, the demand for efficient, reliable, and accessible transport networks is at an all-time high. Among the various modes of transit, bus transportation remains one of the most cost-effective and widely utilized methods globally.

However, managing a vast network of buses, coordinating conductor schedules, processing thousands of passenger details, and maintaining accurate ticketing records poses a massive administrative challenge. Traditional manual systems, reliant on paper ledgers or disorganized spreadsheets, are highly susceptible to human error, data redundancy, and agonizingly slow retrieval times. These inefficiencies result in poor customer service, revenue leakage, and logistical nightmares for transport authorities.

To overcome these hurdles, the **Bus Reservation System** was conceptualized and developed. It is a full-stack, database-driven web application designed specifically to serve as a centralized hub for transport management. The primary objective is to replace archaic manual processes with a streamlined digital workflow that ensures data accuracy, accelerates transaction processing, and provides insightful reports for better decision-making.

This project heavily emphasizes fundamental and advanced concepts of Database Management Systems (DBMS). The core of the application is a deeply analyzed, fully normalized relational database managed by MySQL. By implementing strict constraints, primary and foreign key relationships, and optimized query structures, the system guarantees high performance and data integrity.

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Currently, many regional transport operators lack a unified digital platform to manage their day-to-day operations. The absence of a centralized database leads to the following critical issues:

- **Data Inconsistency:** Passenger information, ticket records, and bus schedules are often stored in disparate, unlinked files, leading to conflicting data.
- **Redundancy:** The same passenger or conductor details might be entered multiple times across different logs, wasting storage space and updating effort.
- **Lack of Real-time Tracking:** Administrators cannot instantly ascertain the total number of bookings, available buses, or conductor assignments without manual counting.
- **Poor Reporting:** Generating daily or weekly manifests (e.g., matching passengers to a specific bus route) is incredibly time-consuming and error-prone.

2.2 Objectives

The primary goal of this Capstone Project is to design, implement, and deploy a comprehensive relational database solution to alleviate the aforementioned problems. The specific objectives include:

- **Relational Modeling:** To architect a structurally sound Entity-Relationship (ER) model encompassing Passengers, Buses, Conductors, and Tickets.
- **Data Normalization:** To normalize the database schema up to the Third Normal Form (3NF) to eliminate insertion, update, and deletion anomalies.
- **Interactive UI Development:** To build a responsive, user-friendly frontend dashboard (using HTML/CSS/JS) that allows operators to seamlessly perform CRUD operations.
- **Backend API Integration:** To develop a Node.js server that acts as the intermediary, securely processing API requests and executing SQL queries dynamically.
- **Advanced SQL Utilization:** To implement complex SQL concepts such as multi-table Joins, Views, and Aggregate functions to generate unified operational reports.

3. SYSTEM ANALYSIS

System Analysis involves a detailed study of the various requirements the proposed system must meet to be considered successful. These requirements are broadly categorized into Functional and Non-Functional requirements.

3.1 Functional Requirements

Functional requirements define the specific behaviors and features the system must support:

- **Dashboard Analytics:** The system must dynamically aggregate and display the total count of registered passengers, operational buses, employed conductors, and booked tickets.
- **Passenger Management:** The system shall allow administrators to register new passengers, capture their demographics (Name, Age, Gender, Contact), update existing profiles, and remove obsolete records.
- **Bus Management:** Administrators must be able to add new fleet assets, specifying unique bus numbers, types (AC, Non-AC, Sleeper), total seating capacity, source, destination, and precise departure/arrival schedules.
- **Conductor Management:** The system requires tracking of conductor profiles, including contact information and years of professional experience, to facilitate future scheduling.
- **Ticket Management:** The core transaction module must record ticket issuance, linking a unique ticket number to a booking date, journey date, assigned seat, and the total fare collected.
- **Comprehensive Reporting:** The system must synthesize data across multiple entities to generate readable reports, such as "Passenger Ticket Details" and "Conductor Bus Assignments," utilizing predefined SQL Views.

3.2 Non-Functional Requirements

Non-functional requirements outline the quality attributes, performance goals, and constraints of the system:

- **Performance and Responsiveness:** The backend API must process standard CRUD requests in minimal time (typically under 500ms). The frontend must update the Document Object Model (DOM) asynchronously using Fetch API to avoid full page reloads, ensuring a fluid user experience.
- **Usability:** The UI is designed with a modern, high-contrast dark theme (incorporating primary accent colors like Neon Green `#32CD32`) to reduce eye strain for operators working long hours and to provide clear visual hierarchies.
- **Data Integrity and Reliability:** The database must strictly enforce ACID (Atomicity, Consistency, Isolation, Durability) properties. Referential integrity is maintained through Foreign Keys with `ON DELETE CASCADE` rules, ensuring no orphaned records exist if a parent entity is removed.
- **Security:** All database interactions from the Node.js server must utilize parameterized queries (Prepared Statements) to mitigate the risk of SQL Injection attacks.

4. ER DIAGRAM

The Entity-Relationship (ER) model is the conceptual foundation of the database. It illustrates the logical structure by defining entities, their attributes, and the intricate relationships connecting them.

In the Bus Reservation System, four primary entities were identified: **Passenger**, **Bus**, **Conductor**, and **Ticket**. The relationships map how these entities interact in a real-world scenario (e.g., Passengers *Buy* Tickets, Conductors *Have* Buses).

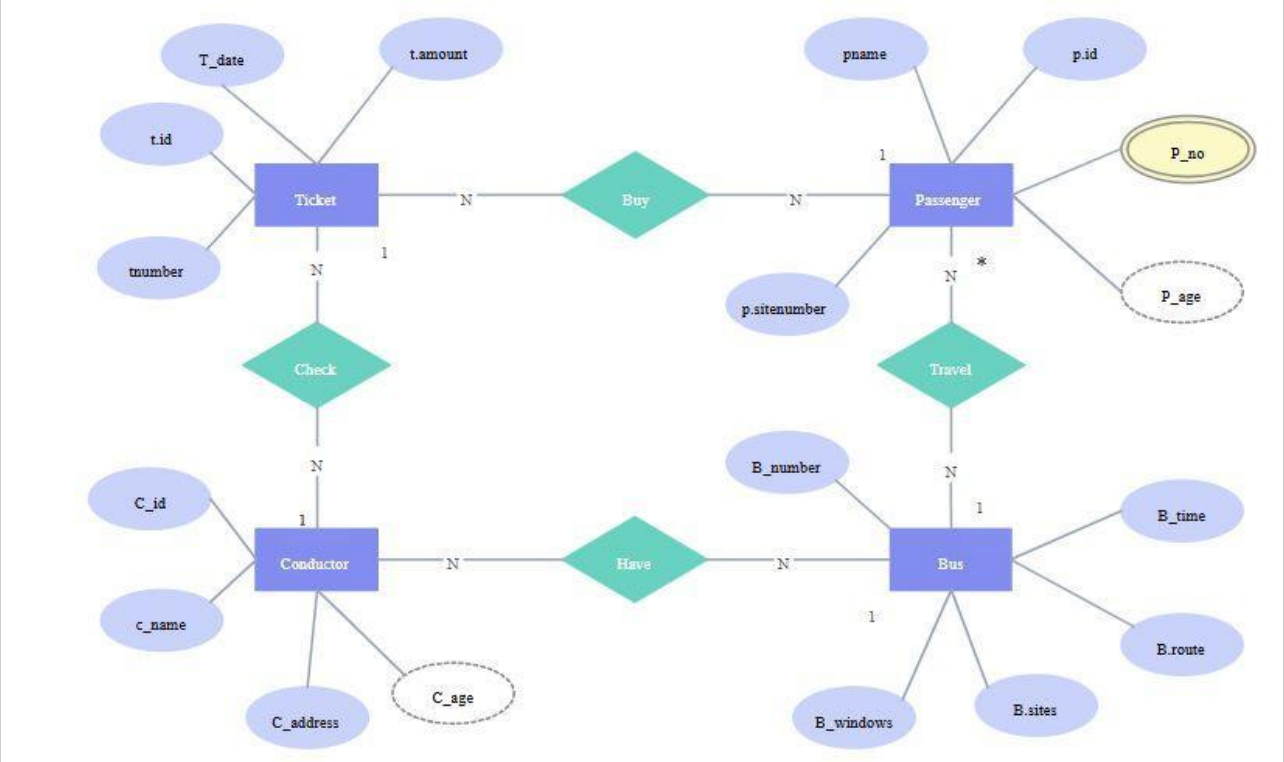

Figure 4.1: Conceptual ER Diagram illustrating entities, attributes, and relationships.

5. RELATIONAL SCHEMA

The Relational Schema translates the conceptual ER diagram into a logical framework consisting of tables (relations), columns (attributes), and constraints (Primary and Foreign Keys).

To resolve the many-to-many relationships present in the ER model, junction (or associative) tables were introduced. This ensures compliance with relational database standards.

Core Entity Tables

- **passengers** (p_id, first_name, last_name, gender, age, phone, email, address)
- **buses** (b_id, *b_number (UNIQUE)*, bus_name, bus_type, total_seats, source, destination, departure_time, arrival_time)
- **conductors** (c_id, first_name, last_name, phone, email, experience_years)
- **tickets** (t_id, *ticket_number (UNIQUE)*, booking_date, journey_date, seat_number, fare)

Junction Tables (Relationship Mapping)

- **passenger_ticket** (p_id, t_id)
  Foreign Keys referencing passengers(p_id) and tickets(t_id).
- **passenger_bus** (p_id, b_number)
  Foreign Keys referencing passengers(p_id) and buses(b_number).
- **conductor_bus** (c_id, b_number)
  Foreign Keys referencing conductors(c_id) and buses(b_number).
- **conductor_ticket** (c_id, t_id)
  Foreign Keys referencing conductors(c_id) and tickets(t_id).

6. NORMALIZATION

Normalization is a systematic approach to decomposing tables to eliminate data redundancy and undesirable characteristics like Insertion, Update, and Deletion anomalies. The database designed for this project strictly adheres to normalization principles up to the Third Normal Form (3NF).

6.1 First Normal Form (1NF)

A relation is in 1NF if every attribute in that relation is single-valued (atomic). There should be no repeating groups or arrays within a single cell.

Application in Project: In the `passengers` table, the passenger's name is split into `first_name` and `last_name` to ensure atomicity. Similarly, phone numbers and emails are stored in distinct, single-valued columns. We do not store multiple ticket numbers in a single column for a passenger; instead, the many-to-many relationship is handled externally.

6.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the entire primary key. This primarily concerns tables with composite primary keys.

Application in Project: The core tables (like `buses`, `conductors`) use a single-column surrogate primary key (e.g., `b_id`, `c_id`), inherently satisfying 2NF. For our junction tables, such as `passenger_ticket (p_id, t_id)`, the table only contains the composite key mapping the relationship. Any specific details about the ticket (like fare) are stored in the `tickets` table, not the junction table, ensuring no partial dependencies exist.

6.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there is no transitive dependency for non-prime attributes. This means every non-key attribute must depend directly on the primary key, and not on another non-key attribute.

Application in Project: In the `tickets` table, attributes like `fare` and `journey_date` depend strictly on the `t_id` (or the unique `ticket_number`). If a passenger's age were placed in the ticket table, it would be a transitive dependency (Ticket -> Passenger -> Age). By maintaining strictly separated tables for Passengers and Tickets, connected only via the `passenger_ticket` junction table, the schema successfully achieves 3NF, ensuring data anomalies are completely eradicated.

7. DATABASE DESIGN & DATA DICTIONARY

This chapter provides a detailed breakdown of the internal structure of the core database tables utilized in the MySQL backend.

7.1 Table: passengers

Attribute Name	Data Type	Constraints	Description
p_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the passenger.
first_name	VARCHAR(50)	NOT NULL	Passenger's given name.
last_name	VARCHAR(50)	NOT NULL	Passenger's family name.
gender	ENUM	('Male','Female','Other'), NOT NULL	Gender specification.
age	INT	NOT NULL	Age of the passenger.
phone	VARCHAR(15)	UNIQUE, NOT NULL	Contact phone number.
email	VARCHAR(100)	UNIQUE	Email address.
address	VARCHAR(255)		Residential or mailing address.

7.2 Table: buses

Attribute Name	Data Type	Constraints	Description
b_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique internal ID.
b_number	VARCHAR(20)	UNIQUE, NOT NULL	

			Registration/License plate number.
bus_name	VARCHAR(100)	NOT NULL	Commercial name or model.
bus_type	ENUM	NOT NULL	AC, Non-AC, Sleeper, Semi-Sleeper.
total_seats	INT	NOT NULL	Maximum seating capacity.
source	VARCHAR(100)	NOT NULL	Origin city/station.
destination	VARCHAR(100)	NOT NULL	Destination city/station.
departure_time	TIME	NOT NULL	Scheduled departure.
arrival_time	TIME	NOT NULL	Scheduled arrival.

7.3 Table: tickets

Attribute Name	Data Type	Constraints	Description
t_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique internal ID.
ticket_number	VARCHAR(20)	UNIQUE, NOT NULL	Alphanumeric ticket code.
booking_date	DATE	NOT NULL	Date the ticket was issued.
journey_date	DATE	NOT NULL	Date of travel.
seat_number	VARCHAR(10)	NOT NULL	Allocated seat number.
fare	DECIMAL	NOT NULL	Cost of the ticket.

8. SQL IMPLEMENTATION

The backend logic is driven by raw SQL scripts crafted to establish the database schema, apply constraints, and ensure high referential integrity. Below are the key Data Definition Language (DDL) implementations used in the project.

8.1 Creating Core Tables

```
-- Passengers Table
CREATE TABLE passengers (
    p_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    gender ENUM('Male', 'Female', 'Other') NOT NULL,
    age INT NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE,
    address VARCHAR(255)
);

-- Buses Table
CREATE TABLE buses (
    b_id INT AUTO_INCREMENT PRIMARY KEY,
    b_number VARCHAR(20) UNIQUE NOT NULL,
    bus_name VARCHAR(100) NOT NULL,
    bus_type ENUM('AC', 'Non-AC', 'Sleeper', 'Semi-Sleeper') NOT NULL,
    total_seats INT NOT NULL,
    source VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    departure_time TIME NOT NULL,
    arrival_time TIME NOT NULL
);
```

8.2 Junction Tables with Cascading Constraints

To maintain relational integrity, Foreign Keys with `ON DELETE CASCADE` are utilized. This ensures that deleting a passenger automatically clears their ticket mappings.

```
CREATE TABLE passenger_ticket (
    p_id INT,
    t_id INT,
    PRIMARY KEY (p_id, t_id),
    FOREIGN KEY (p_id) REFERENCES passengers(p_id) ON DELETE CASCADE,
    FOREIGN KEY (t_id) REFERENCES tickets(t_id) ON DELETE CASCADE
);

CREATE TABLE conductor_bus (
    c_id INT,
    b_number VARCHAR(20),
    PRIMARY KEY (c_id, b_number),
    FOREIGN KEY (c_id) REFERENCES conductors(c_id) ON DELETE CASCADE,
    FOREIGN KEY (b_number) REFERENCES buses(b_number) ON DELETE CASCADE
);
```

9. VIEWS, JOINS & REPORTS

To fulfill complex reporting requirements without writing massive nested queries repeatedly within the application backend, SQL Views were implemented. These Views heavily utilize `INNER JOIN` operations to flatten related data from multiple tables.

9.1 Passenger Ticket Details View

This view merges data from the `passengers` table, the junction table `passenger_ticket`, and the `tickets` table to show which passenger holds which ticket.

```
CREATE VIEW passenger_ticket_details AS
SELECT
    p.p_id,
    CONCAT(p.first_name, ' ', p.last_name) AS passenger_name,
    t.ticket_number,
    t.seat_number,
    t.fare
FROM passengers p
INNER JOIN passenger_ticket pt
    ON p.p_id = pt.p_id
INNER JOIN tickets t
    ON pt.t_id = t.t_id;
```

9.2 Passenger Bus Details View

This view links passengers directly to the operational details of the bus they are scheduled to travel on.

```
CREATE VIEW passenger_bus_details AS
SELECT
    p.p_id,
    CONCAT(p.first_name, ' ', p.last_name) AS passenger_name,
    b.b_number,
    b.bus_name,
    b.bus_type,
    b.source,
    b.destination,
    b.departure_time,
    b.arrival_time
FROM passengers p
INNER JOIN passenger_bus pb
    ON p.p_id = pb.p_id
INNER JOIN buses b
    ON pb.b_number = b.b_number;
```

10. FRONTEND, BACKEND & DEPLOYMENT

10.1 Frontend Architecture

The client side of the application is an intuitive Single Page Application (SPA) designed with:

- **HTML5:** Defines the semantic structure of the dashboard, navigation bars, and data-entry forms.
- **CSS3:** Implements a sophisticated dark theme utilizing CSS variables (e.g., `--bg:#0D0D0D`, `--primary:#32CD32`). It uses CSS Grid and Flexbox for responsive card layouts.
- **JavaScript:** Handles DOM manipulation, toggles visibility of sections based on navigation clicks, captures form data, and uses the `fetch()` API to interact with the backend server asynchronously.

10.2 Backend Architecture

The backend is developed purely in **Node.js** without the use of heavy frameworks like Express, showcasing a fundamental understanding of network protocols.

- **Server Module (server.js):** Utilizes the native `http` module to create a server listening on port 3000. It manually parses URL paths, handles CORS headers, processes JSON request bodies, and routes traffic to specific database logic functions.
- **Database Integration (db.js):** Employs the `mysql2` driver to establish a robust connection pool to the MySQL database using Environment Variables (dotenv) for security.

10.3 Deployment Strategy

The project adopts a modern cloud deployment architecture to ensure high availability. The application is containerized/hosted using Platform-as-a-Service (PaaS) providers like Railway.

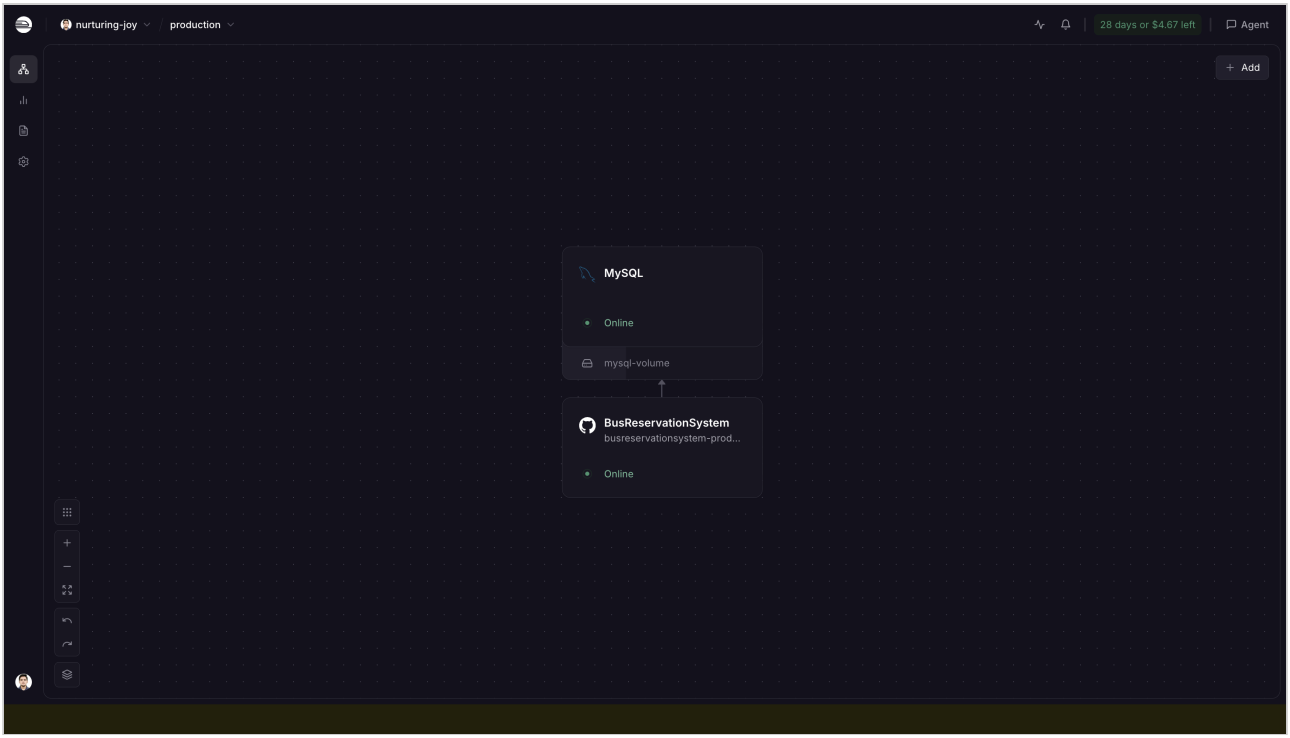

Figure 10.1: Deployment pipeline showcasing the Web Service communicating with a detached MySQL Volume via environment variables.

11. RESULTS & SCREENSHOTS

The implemented system successfully achieves all functional goals, providing administrators with a seamless interface to manage transport logistics.

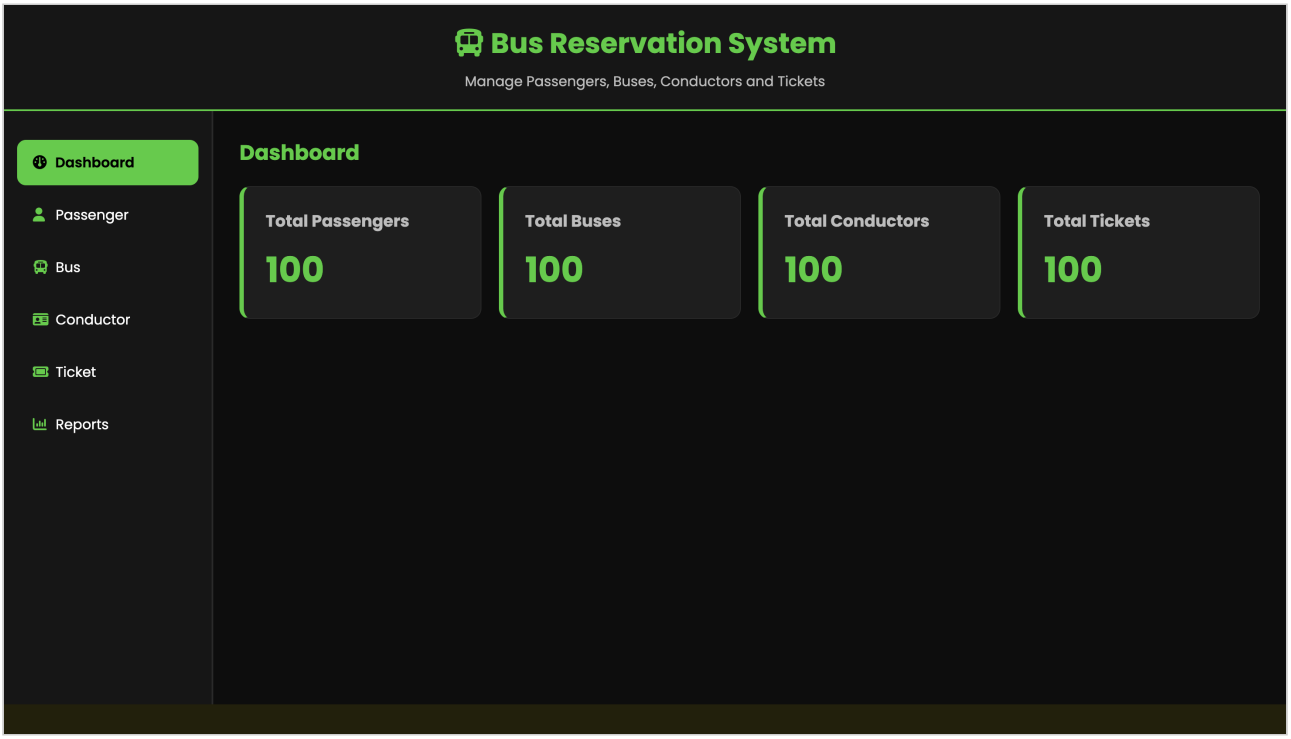

Figure 11.1: The main Admin Dashboard displaying real-time aggregated metric counts fetched dynamically from the database.

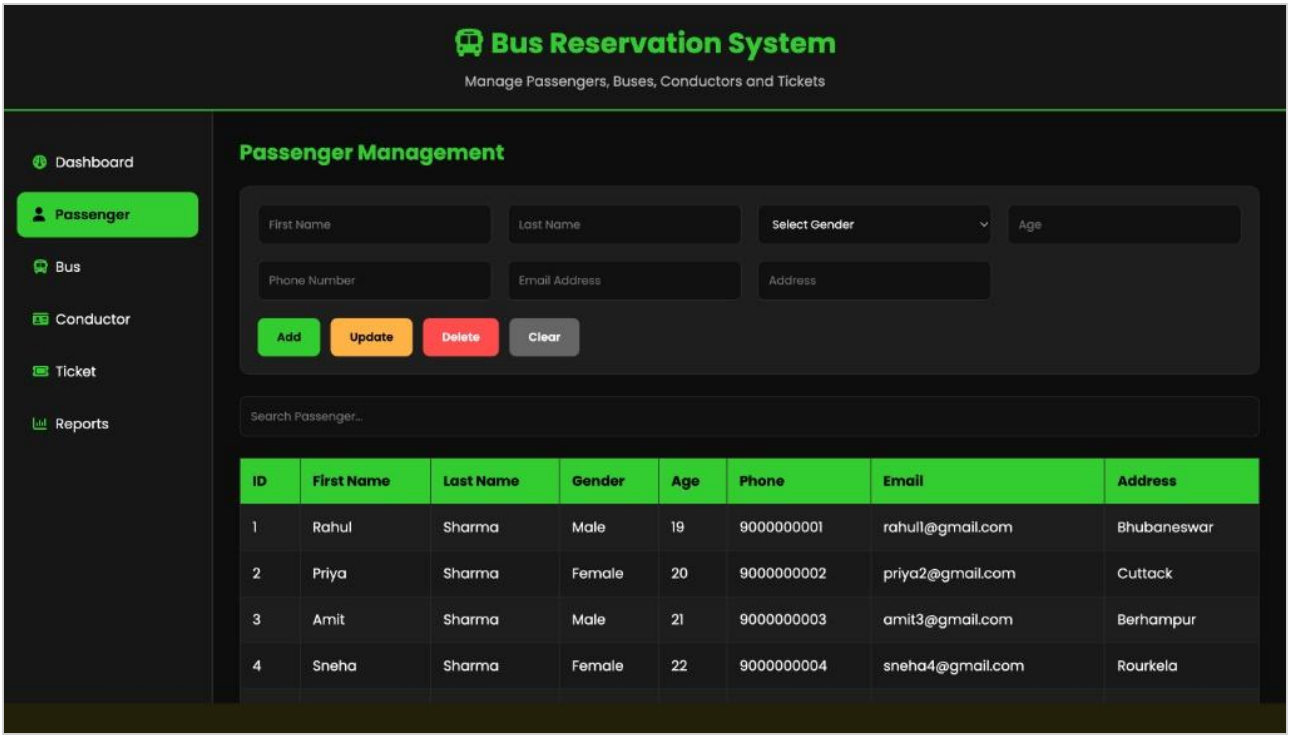

Figure 11.2: Passenger Management Module featuring a data entry form, search functionality, and a tabulated view of records.

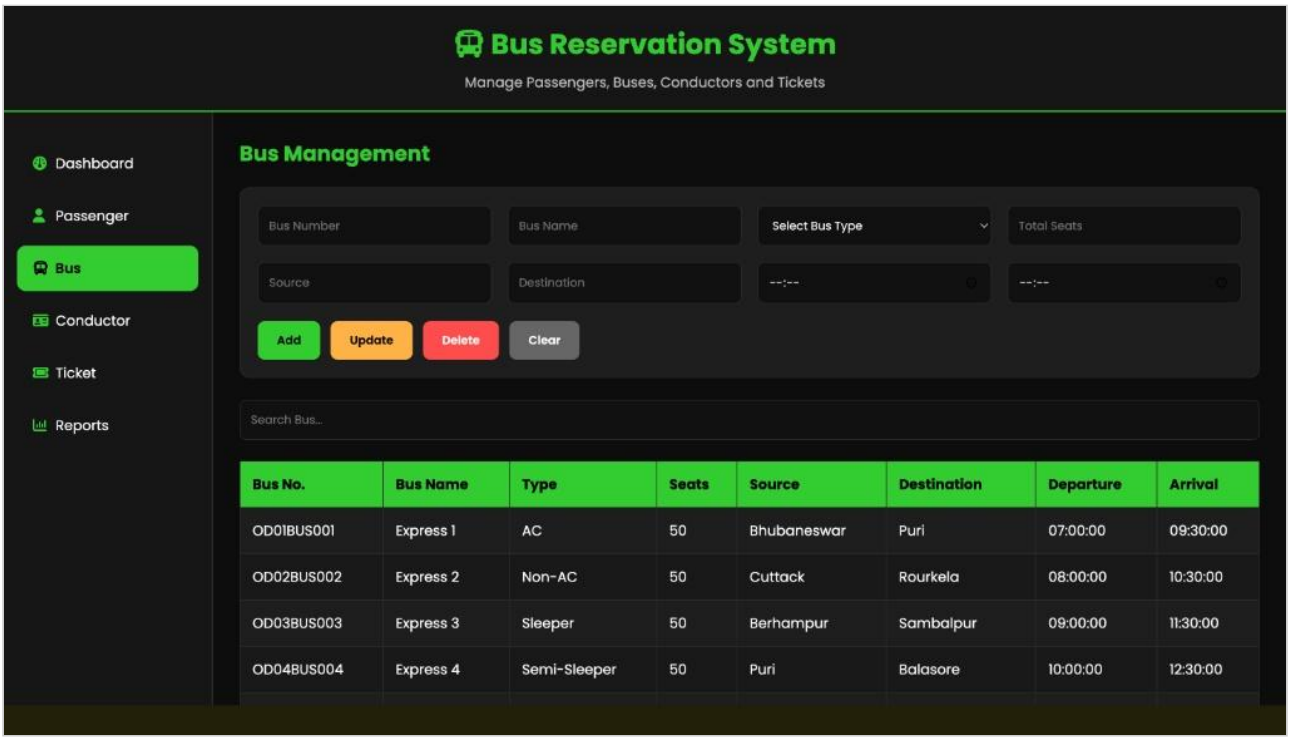

Figure 11.3: Bus Management Module showcasing route planning and capacity tracking capabilities.

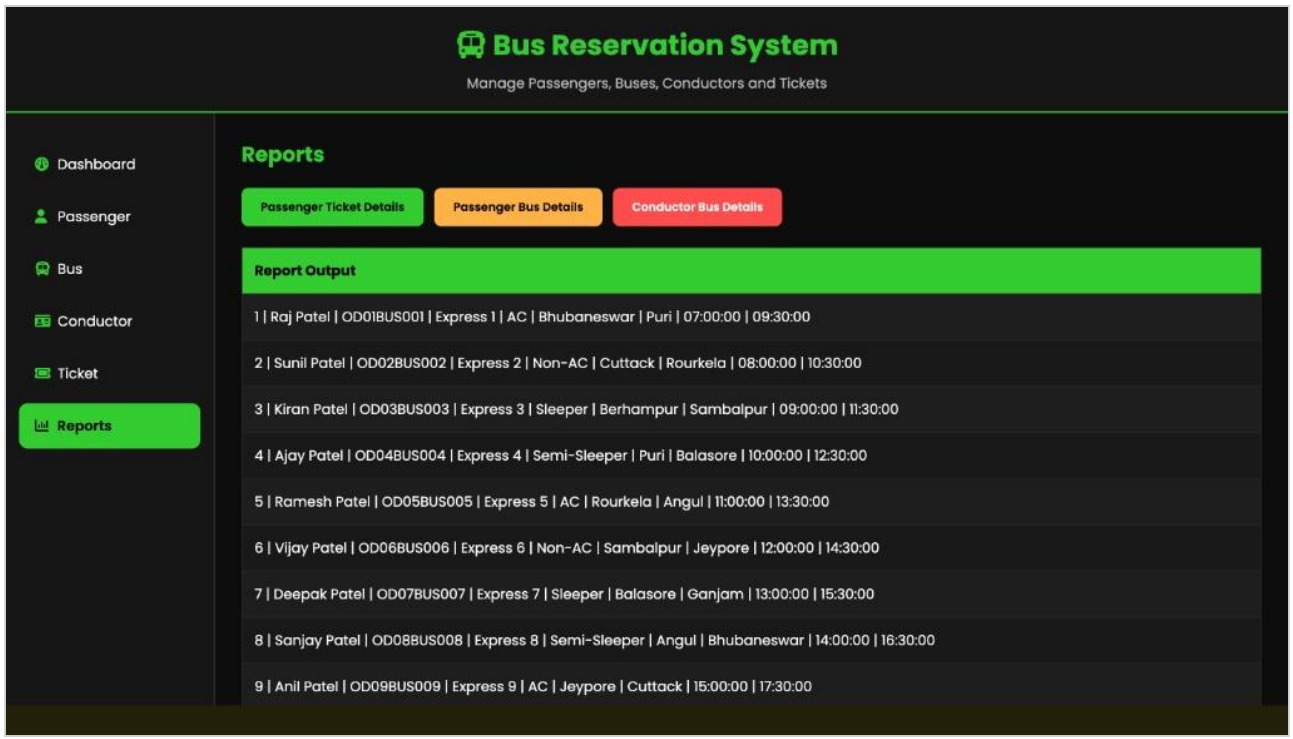

Figure 11.4: Reports Module outputting flattened, human-readable data generated by the backend SQL Views.

12. CONCLUSION & FUTURE SCOPE

12.1 Conclusion

The Bus Reservation System capstone project successfully demonstrates the end-to-end development of a robust database application. By meticulously transitioning from conceptual ER modeling to logical schema design, and enforcing normalization up to the Third Normal Form (3NF), the project ensures absolute data integrity and operational efficiency.

The separation of concerns between the dynamic HTML/CSS frontend, the lightweight Node.js API backend, and the highly relational MySQL database mimics professional software engineering workflows. The system completely eradicates the redundancies associated with manual record-keeping, allowing administrators to execute complex logistical workflows—from ticket assignment to conductor scheduling—with speed and accuracy.

12.2 Future Scope

While the current system covers core administrative requirements, several enhancements can be integrated in future iterations:

- **User Authentication:** Implementing JWT (JSON Web Tokens) for role-based access control (Admin vs. Standard Operator).
- **Payment Gateway Integration:** Enabling the backend to process actual financial transactions and record real-time billing history.
- **Graphical Seat Selection:** Developing an interactive frontend widget allowing users to click and select specific seats on a graphical bus layout.
- **Automated Notifications:** Integrating an email or SMS API to dispatch automated ticket confirmations to passengers upon booking.